

Chapter 9. Pods and Namespaces

The most important primitive in the Kubernetes API is the Pod. A Pod lets you run a containerized application. In practice, you'll often encounter a one-to-one mapping between a Pod and a container; however, there are use cases that benefit from declaring more than one container in a single Pod.

Multi-container Pods are useful for tightly coupled processes that need to share resources like storage volumes and network namespace, fulfilling use cases such as sidecar patterns (log collection, monitoring agents), adapter patterns (standardizing output formats), ambassador patterns (proxy connections), and init containers (performing setup tasks before main containers start). See the [Kubernetes blog](#) for an overview.

In addition to running a container, a Pod can consume other services like storage, configuration data, and much more. Therefore, think of a Pod as a wrapper for running containers while at the same time being able to mix in cross-cutting and specialized Kubernetes features.

COVERAGE OF CURRICULUM OBJECTIVES

The curriculum doesn't explicitly mention coverage of Pods and namespaces. However, you will definitely need to understand those primitives as they are essential for running workloads in Kubernetes.

Working with Pods

In this chapter, we will look at working with a Pod running only a single container. I'll discuss all important `kubectl` commands for creating, modifying, interacting, and deleting using imperative and declarative approaches.

Creating Pods

The Pod definition needs to state an image for every container. Upon creating the Pod object, imperatively or declaratively, the scheduler will assign the Pod to a node, and the container runtime engine will check if the

container image already exists on that node. If the image doesn't exist yet, the engine will download it from a container image registry defined as default by the container runtime. As soon as the image exists on the node, the container is instantiated and will run. [Figure 9-1](#) demonstrates the execution flow.

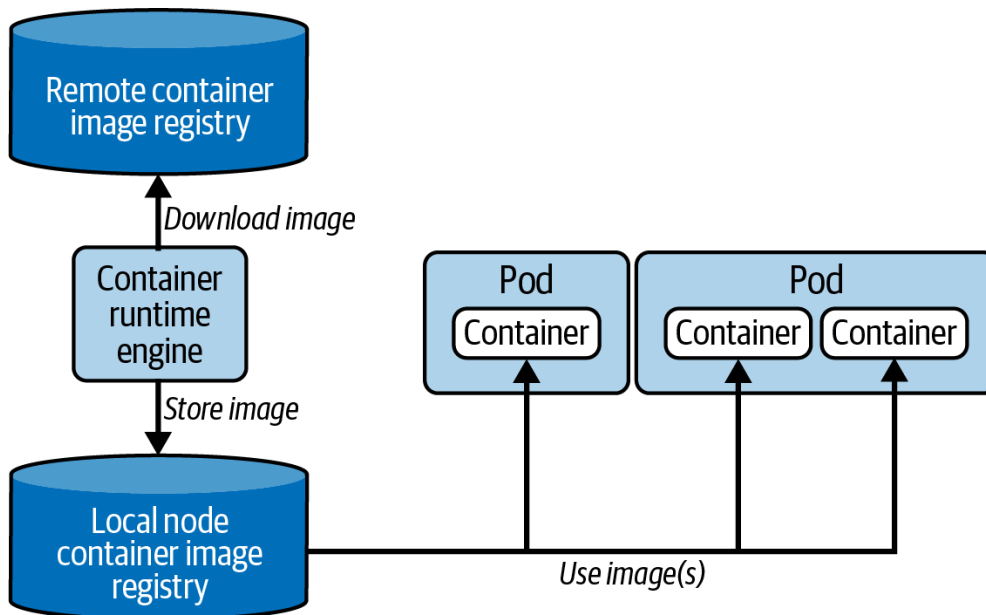


Figure 9-1. Container Runtime Interface interaction with container images

The `run` command is the central entry point for creating Pods imperatively. Let's talk about its usage and the most important command-line options you should memorize and practice. Say you wanted to run a [Hazelcast instance](#) inside of a Pod. The container should use the latest [Hazelcast image](#), expose port 5701, and define an environment variable. In addition, you also want to assign two labels to the Pod. The following imperative command combines this information and does not require any further editing of the live object:

```
$ kubectl run hazelcast --image=hazelcast/hazelcast:5.1.7 \
  --port=5701 --env="DNS_DOMAIN=cluster" --labels="app=hazelcast,env=prod"
```

The `run` command offers a wealth of command-line options. Execute the `kubectl run --help` or refer to the [Kubernetes documentation](#) for a broad overview. For the exam, you'll not need to understand every command. [Table 9-1](#) lists the most commonly used options.

Table 9-1. Important `kubectl run` command-line options

Option	Example value	Description
<code>--image</code>	<code>hazelcast/hazelcast:5.1.7</code>	The image for the container to run.
<code>--port</code>	<code>5701</code>	The port that this container exposes.
<code>--rm</code>	<code>true</code>	Deletes the Pod after command in the container finishes. See “Creating a Temporary Pod” for more information.
<code>--env</code>	<code>DNS_DOMAIN=cluster</code>	The environment variables to set in the container.
<code>--labels</code>	<code>app=hazelcast,env=prod</code>	A comma-separated list of labels to apply to the Pod.

Some developers are more used to creating Pods from a YAML manifest. You’re probably already accustomed to the declarative approach because it provides version-controllable, reproducible infrastructure as code that can be stored in Git, enabling GitOps workflows, peer review, and roll-back capabilities. You can express the same configuration for the Hazelcast Pod by opening the editor, copying a Pod YAML code snippet from the Kubernetes online documentation, and modifying it to your needs. [Example 9-1](#) shows the Pod manifest saved in the file *pod.yaml*:

Example 9-1. Pod YAML manifest

```

apiVersion: v1
kind: Pod
metadata:
  name: hazelcast
  labels:
    app: hazelcast
    env: prod
spec:
  containers:
  - name: hazelcast
    image: hazelcast/hazelcast:5.1.7
    env:
      - name: DNS_DOMAIN
        value: cluster
    ports:
      - containerPort: 5701

```

❶ Assigns the name of `hazelcast` to the Pod

❷ Specifies labels to the Pod

- ❸ Declares the container image to be executed in the container of the Pod
- ❹ Injects one or many environment variables to the container
- ❺ Number of port to expose on the Pod's IP address

Creating the Pod from the manifest is straightforward. Simply use the `create` or `apply` command, as shown here and explained in [“Managing Objects”](#):

```
$ kubectl apply -f pod.yaml
pod/hazelcast created
```

Listing Pods

Now that you have created a Pod, you can further inspect its runtime information. The `kubectl` command offers a command for listing all Pods running in the cluster: `get pods`. The following command renders the Pod named `hazelcast`:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
hazelcast	1/1	Running	0	17s

Real-world Kubernetes clusters can run hundreds of Pods at the same time. If you know the name of the Pod of interest, it's often easier to query by name. You would still see only a single Pod:

```
$ kubectl get pods hazelcast
```

NAME	READY	STATUS	RESTARTS	AGE
hazelcast	1/1	Running	0	17s

Since we're covering basic Pod listing, it's worth mentioning `kubectl get pods | grep <pattern>` as a practical alternative to exact name lookups, especially when searching for Pods created by Deployments where names include generated suffixes—for example, `kubectl get pods | grep nginx` finds all NGINX-related Pods regardless of their full generated names.

Even better is using label selectors like `kubectl get pods -l app=hazelcast`, which queries Pods by their labels rather than names, making it more precise and Kubernetes-native than `grep` while also working across namespaces with `-A` or `--all-namespaces`.

Pod Lifecycle Phases

Because Kubernetes is a state engine with asynchronous control loops, it's possible that the status of the Pod doesn't show a `Running` status right away when listing the Pods. It usually takes a couple of seconds to retrieve the image and start the container. Upon Pod creation, the object goes through several [lifecycle phases](#), as shown in [Figure 9-2](#).

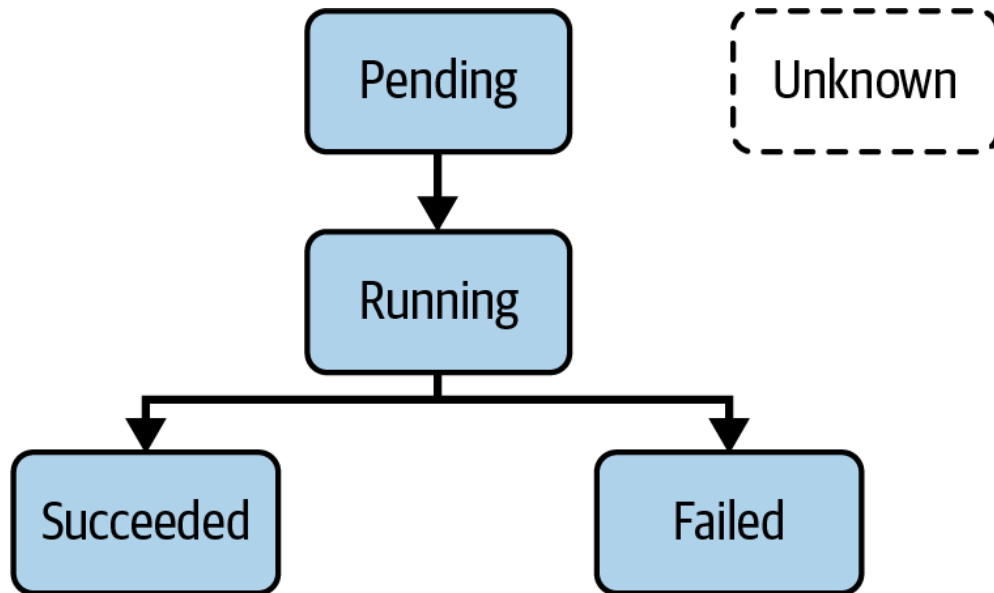


Figure 9-2. Pod lifecycle phases

Understanding the implications of each phase is important, as it gives you an idea about the operational status of a Pod. For example, during the exam you may be asked to identify a Pod with an issue and further debug the object. [Table 9-2](#) describes all Pod lifecycle phases.

Table 9-2. Pod lifecycle phases

Option	Description
Pending	The Pod has been accepted by the Kubernetes system, but one or more of the container images has not been created.
Running	At least one container is still running or is in the process of starting or restarting.
Succeeded	All containers in the Pod terminated successfully.
Failed	Containers in the Pod terminated, at least one failed with an error.
Unknown	The state of the Pod could not be obtained.

The Pod lifecycle phases should not be confused with container states within a Pod. Containers can have one of the three possible states:

Waiting, Running, and Terminated. You can read more about container states in the [Kubernetes documentation](#).

Container-Level Restarts

Every Pod provides container-level restart capabilities. If a container fails, the kubelet cluster component will restart it based on its configured [restart policy](#).

The restart policy of a Pod can be set using the attribute `spec.restartPolicy`. Possible values for this attribute include `Always`, `OnFailure`, and `Never`, as shown in [Table 9-3](#). The default value is `Always` if the attribute has not been set explicitly.

Table 9-3. Container-level restart options

Option	Description
<code>Always</code>	Automatically restarts the container after any termination
<code>OnFailure</code>	Only restarts the container if it exits with an error (non-zero exit status)
<code>Never</code>	Does not automatically restart the terminated container

[Example 9-2](#) shows the usage of the attribute in a Pod manifest.

Example 9-2. Setting a Pod's restart policy

```
apiVersion: v1
kind: Pod
metadata:
  name: hazelcast
spec:
  containers:
  - name: hazelcast
    image: hazelcast/hazelcast:5.1.7
    restartPolicy: Never
```

The attribute applies to a Pod's [application containers](#) and [init containers](#) (both regular init containers and sidecar containers). Sidecar containers require `restartPolicy: Always` to be explicitly set.

Rendering Pod Details

The rendered table produced by the `get` command provides high-level information about a Pod. But what if you needed a deeper look at the details? The `describe` command can help:

```

$ kubectl describe pods hazelcast
Name:          hazelcast
Namespace:     default
Priority:      0
PriorityClassName: <none>
Node:         docker-desktop/192.168.65.3
Start Time:   Wed, 20 May 2020 19:35:47 -0600
Labels:       app=hazelcast
              env=prod
Annotations:  <none>
Status:       Running
IP:           10.1.0.41
Containers:
  ...
Events:
  ...

```

The terminal output contains the metadata information of a Pod, the containers it runs, and the event log, such as failures when the Pod was scheduled. The example output has been condensed to show just the metadata section. You can expect the output to be lengthy.

There's a way to be more specific about the information you want to render. You can combine the `describe` command with a Unix `grep` command if you want to identify the image for running in the container:

```

$ kubectl describe pods hazelcast | grep Image:
Image:          hazelcast/hazelcast:5.1.7

```

Accessing Logs of a Pod

As application developers, we know very well what to expect in the log files produced by the application we implemented. Runtime failures may occur when operating an application in a container. The `logs` command downloads the log output of a container. The following output indicates that the Hazelcast server started up successfully:

```

$ kubectl logs hazelcast
...
May 25, 2020 3:36:26 PM com.hazelcast.core.LifecycleService
INFO: [10.1.0.46]:5701 [dev] [4.0.1] [10.1.0.46]:5701 is STARTED

```

It's very likely that more log entries will be produced as soon as the container receives traffic from end users. You can stream the logs with the

command-line option `-f`. This option is helpful if you want to see logs in real time.

Kubernetes tries to restart a container under certain conditions, such as if the image cannot be resolved on the first try. Upon a container restart, you won't have access to the logs of the previous container; the `logs` command renders the logs only for the current container. However, you can still get back to the logs of the previous container by adding the `-p` command-line option. You may want to use that option to identify the root cause that triggered a container restart.

Executing a Command in Container

Some situations require you to get the shell to a running container and explore the filesystem. Maybe you want to inspect the configuration of your application or debug its current state. You can use the `exec` command to open a shell in the container to explore it interactively, as follows:

```
$ kubectl exec -it hazelcast -- /bin/sh
# ...
```

Notice that you do not have to provide the resource type. This command only works for a Pod. The two dashes (`--`) separate the `exec` command and its options from the command you want to run inside of the container.

It's also possible to execute a single command inside of a container. Say you wanted to render the environment variables available to containers without having to be logged in. Just remove the interactive flag `-it` and provide the relevant command after the two dashes:

```
$ kubectl exec hazelcast -- env
...
DNS_DOMAIN=cluster
```

Creating a Temporary Pod

The command executed inside of a Pod—usually an application implementing business logic—is meant to run infinitely. Once the Pod has been created, it will stick around. Under certain conditions, you'll want to execute a command in a Pod just for troubleshooting. This use case doesn't require a Pod object to run beyond the execution of the command. That's where temporary Pods come into play.

The `run` command provides the flag `--rm`, which will automatically delete the Pod after the command running inside of it finishes. Say you want to render all environment variables using `env` to see what's available inside of the container. The following command achieves exactly that:

```
$ kubectl run busybox --image=busybox:1.36.1 --rm -it --restart=Never --
...
HOSTNAME=busybox
pod "busybox" deleted
```

The last message rendered in the output clearly states that the Pod was deleted after command execution.

Using a Pod's IP Address for Network Communication

Every Pod is assigned an IP address upon creation. You can inspect a Pod's IP address by using the `-o wide` command-line option for the `get pod` command or by describing the Pod. The IP address of the Pod in the following console output is `10.244.0.5`:

```
$ kubectl run nginx --image=nginx:1.25.1 --port=80
pod/nginx created
$ kubectl get pod nginx -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          \
NOMINATED     NODE    READINESS GATES
nginx         1/1     Running   0           37s   10.244.0.5    minikube     \
<none>        <none>
$ kubectl get pod nginx -o yaml
...
status:
  podIP: 10.244.0.5
...
```

The IP address assigned to a Pod is unique across all nodes and namespaces. This is achieved by assigning a dedicated subnet to each node when registering it. When creating a new Pod on a node, the IP address is leased from the assigned subnet. This is handled by the networking lifecycle manager kube-proxy along with the Domain Name System (DNS) and the Container Network Interface (CNI).

You can easily verify the behavior by creating a temporary Pod that calls the IP address of another Pod using the command-line tool `curl` or `wget`:

```
$ kubectl run busybox --image=busybox:1.36.1 --rm -it --restart=Never \
-- wget 10.244.0.5:80
Connecting to 10.244.0.5:80 (10.244.0.5:80)
saving to 'index.html'
index.html          100% |*****|      615   0:00:00
'index.html' saved
pod "busybox" deleted
```

Configuring Pods

The curriculum expects you to feel comfortable with editing YAML manifests either as files or as live object representations. This section shows you some typical configuration scenarios you may face during the exam. Later chapters will deepen your knowledge by touching on other configuration aspects.

Declaring environment variables

Applications need to expose a way to make their runtime behavior configurable. For example, you may want to inject the URL to an external web service or declare the username for a database connection. Environment variables are a common option to provide this runtime configuration.

AVOID CREATING CONTAINER IMAGES PER ENVIRONMENT

It might be tempting to say, “Hey, let’s create a container image for any target deployment environment we need, including its configuration.” That’s a bad idea. One of the practices of [continuous delivery](#) and the [Twelve-Factor App principles](#) is to build a deployable artifact for a commit just once. In this case, the artifact is the container image. Deviating configuration runtime behavior should be controllable by injecting runtime information when instantiating the container. You can use environment variables to control the behavior as needed.

Defining environment variables in a Pod YAML manifest is relatively easy. Add or enhance the section `env` of a container. Every environment variable consists of a key-value pair, represented by the attributes `name` and `value`. Kubernetes does not enforce or sanitize typical naming conventions for environment variable keys, though it is recommended to follow the standard of using uppercase letters and using the underscore character (`_`) to separate words.

To illustrate a set of environment variables, look at [Example 9-3](#). The code snippet describes a Pod that runs a Java-based application using the Spring Boot framework.

Example 9-3. YAML manifest for a Pod defining environment variables

```
apiVersion: v1
kind: Pod
metadata:
  name: spring-boot-app
spec:
  containers:
  - name: spring-boot-app
    image: springio/gs-spring-boot-docker
    env:
    - name: SPRING_PROFILES_ACTIVE
      value: dev
    - name: VERSION
      value: '1.5.3'
```

The first environment variable, named `SPRING_PROFILES_ACTIVE`, defines a pointer to a so-called *profile*. Spring uses profiles to manage environment-specific properties. Here, we are pointing to the profile that configures the production environment. The environment variable `VERSION` specifies the application version. Its value can be exposed by the running application to display the value in the user interface.

Defining a command with arguments

Many container images already define an `ENTRYPOINT` or `CMD` instruction. The command assigned to the instruction is automatically executed as part of the container startup. For example, the Hazelcast image we used earlier defines the instruction `CMD [ "/opt/hazelcast/start-hazelcast.sh" ]`.

In a Pod definition, you can either redefine the image `ENTRYPOINT` and `CMD` instructions or assign a command to execute for the container if it hasn't been specified by the image. You can provide this information with the help of the `command` and `args` attributes for a container. The `command` attribute overrides the image's `ENTRYPOINT` instruction. The `args` attribute replaces the `CMD` instruction of an image.

Imagine you wanted to provide a command to an image that doesn't provide one yet. As usual, there are two different approaches: imperative and declarative. We'll generate the YAML manifest with the help of the `run` command. The Pod should use the `busybox:1.36.1` image and execute a shell command that renders the current date every 10 seconds in an infinite loop:

```
$ kubectl run mypod --image=busybox:1.36.1 -o yaml --dry-run=client \
> pod.yaml -- /bin/sh -c "while true; do date; sleep 10; done"
```

You can see in the generated but condensed *pod.yaml* file shown in [Example 9-4](#) that the command has been turned into an `args` attribute. Kubernetes specifies each argument on a single line.

Example 9-4. A YAML manifest containing an `args` attribute

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: busybox:1.36.1
    args:
    - /bin/sh
    - -c
    - while true; do date; sleep 10; done
```

You could have achieved the same result by a combination of the `command` and `args` attributes if you were to handcraft the YAML manifest. [Example 9-5](#) shows a different approach.

Example 9-5. A YAML manifest containing `command` and `args` attributes

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: busybox:1.36.1
    command: ["/bin/sh"]
    args: ["-c", "while true; do date; sleep 10; done"]
```

You can quickly verify if the declared command actually does its job. First, create the Pod instance, then tail the logs:

```
$ kubectl apply -f pod.yaml
pod/mypod created
$ kubectl logs mypod -f
```

```
Fri May 29 00:49:06 UTC 2020
Fri May 29 00:49:16 UTC 2020
Fri May 29 00:49:26 UTC 2020
...
```

Deleting a Pod

Sooner or later you'll want to delete a Pod. During the exam, you may be asked to delete a Pod. Or possibly, you made a configuration mistake and want to start the question from scratch:

```
$ kubectl delete pod hazelcast
pod "hazelcast" deleted
```

Keep in mind that Kubernetes tries to delete a Pod *gracefully*. This means that the Pod will try to finish active requests to the Pod to avoid unnecessary disruption to the end user. A graceful deletion operation can take anywhere from 5 to 30 seconds, time you don't want to waste during the exam. See [Chapter 1](#) for more information on how to speed up the process.

An alternative way to delete a Pod is to point the `delete` command to the YAML manifest you used to create it. The behavior is the same:

```
$ kubectl delete -f pod.yaml
pod "hazelcast" deleted
```

To save time during the exam, you can circumvent the grace period by adding the `--now` option to the `delete` command. Avoid using the `--now` flag in production Kubernetes environments.

Working with Namespaces

Namespaces are an API construct used to avoid naming collisions, and they represent a scope for object names. A good use case for namespaces is to isolate the objects by team or responsibility.

NAMESPACES FOR OBJECTS

The content in this chapter demonstrates the use of namespaces for Pod objects. Namespaces are not a concept applicable only to Pods, though. Most object types can be grouped by a namespace.

Most questions in the exam will ask you to execute the command in a specific namespace that has been set up for you. The following sections briefly touch on the basic operations needed to deal with a namespace.

Listing Namespaces

A Kubernetes cluster starts out with a couple of initial namespaces. You can list them with the following command:

```
$ kubectl get namespaces
NAME                STATUS    AGE
default             Active    157d
kube-node-lease     Active    157d
kube-public         Active    157d
kube-system         Active    157d
```

The `default` namespace hosts objects that haven't been assigned to an explicit namespace. Namespaces starting with the prefix `kube-` are not considered end user-namespaces. They have been created by the Kubernetes system.

Creating and Using a Namespace

To create a new namespace, use the `create namespace` command. The following command uses the name `code-red`:

```
$ kubectl create namespace code-red
namespace/code-red created
$ kubectl get namespace code-red
NAME        STATUS    AGE
code-red    Active    16s
```

[Example 9-6](#) shows the corresponding representation as a YAML manifest.

Example 9-6. Namespace YAML manifest

```
apiVersion: v1
kind: Namespace
metadata:
  name: code-red
```

Once the namespace is in place, you can create objects within it. You can do so with the command-line option `--namespace` or its short-form `-n`.

The following commands create a new Pod in the namespace `code-red` and then list the available Pods in the namespace:

```
$ kubectl run pod --image=nginx:1.25.1 -n code-red
pod/pod created
$ kubectl get pods -n code-red
```

NAME	READY	STATUS	RESTARTS	AGE
pod	1/1	Running	0	13s

Setting a Namespace Preference

Providing the `--namespace` or `-n` command-line option for every command is tedious and error-prone. You can set a permanent namespace preference if you know that you need to interact with a specific namespace you are responsible for. The first command shown sets the permanent namespace `code-red`. The second command renders the currently set permanent namespace:

```
$ kubectl config set-context --current --namespace=code-red
Context "minikube" modified.
$ kubectl config view --minify | grep namespace:
    namespace: code-red
```

Subsequent `kubectl` executions do not have to spell out the namespace `code-red`:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
pod	1/1	Running	0	13s

You can always switch back to the `default` namespace or another custom namespace using the `config set-context` command:

```
$ kubectl config set-context --current --namespace=default
Context "minikube" modified.
```

Deleting a Namespace

Deleting a namespace has a cascading effect on the object existing in it. Deleting a namespace will automatically delete its objects:

```
$ kubectl delete namespace code-red
namespace "code-red" deleted
```

```
$ kubectl get pods -n code-red
```

```
No resources found in code-red namespace.
```

Summary

The exam puts a strong emphasis on the concept of a Pod, a Kubernetes primitive responsible for running an application in a container. A Pod can define one or many containers that use a container image. Upon its creation, the container image is resolved and used to bootstrap the application. Every Pod can be further customized with the relevant YAML configuration.

Exam Essentials

Know how to interact with Pods

A Pod runs an application inside of a container. You can check on the status and the configuration of the Pod by inspecting the object with the `kubectl get` or `kubectl describe` commands. Get familiar with the lifecycle phases of a Pod to be able to quickly diagnose errors. The command `kubectl logs` can be used to download the container log information without having to shell into the container. Use the command `kubectl exec` to further explore the container environment, e.g., to check on processes or to examine files.

Understand advanced Pod configuration options

Sometimes you have to start with the YAML manifest of a Pod and then create the Pod declaratively. This could be the case if you wanted to provide environment variables to the container or declare a custom command. Practice different configuration options by copy-pasting relevant code snippets from the Kubernetes documentation.

Practice using a custom namespace

Most questions in the exam will ask you to work within a given namespace. You need to understand how to interact with that namespace from `kubectl` using the options `--namespace` and `-n`. To avoid accidentally working on the wrong namespace, know how to permanently set a namespace.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Create a new Pod named `nginx` running the image `nginx:1.17.10`.
Expose the container port `80`. The Pod should live in the namespace named `j43`.
Get the details of the Pod including its IP address.
Create a temporary Pod that uses the `busybox:1.36.1` image to execute a `wget` command inside of the container. The `wget` command should access the endpoint exposed by the `nginx` container. You should see the HTML response body rendered in the terminal.
Get the logs of the `nginx` container.
Add the environment variables `DB_URL=postgresql://mydb:5432` and `DB_USERNAME=admin` to the container of the `nginx` Pod.
Open a shell for the `nginx` container and inspect the contents of the current directory `ls -l`. Exit out of the container.
2. Create a YAML manifest for a Pod named `loop` in the namespace `j43` that runs the `busybox:1.36.1` image in a container. The container should run the following command: `for i in {1..10}; do echo "Welcome $i times"; done`. Create the Pod from the YAML manifest. What's the status of the Pod?
Edit the Pod named `loop`. Change the command to run in an endless loop. Each iteration should `echo` the current date.
Inspect the events and the status of the Pod `loop`.